

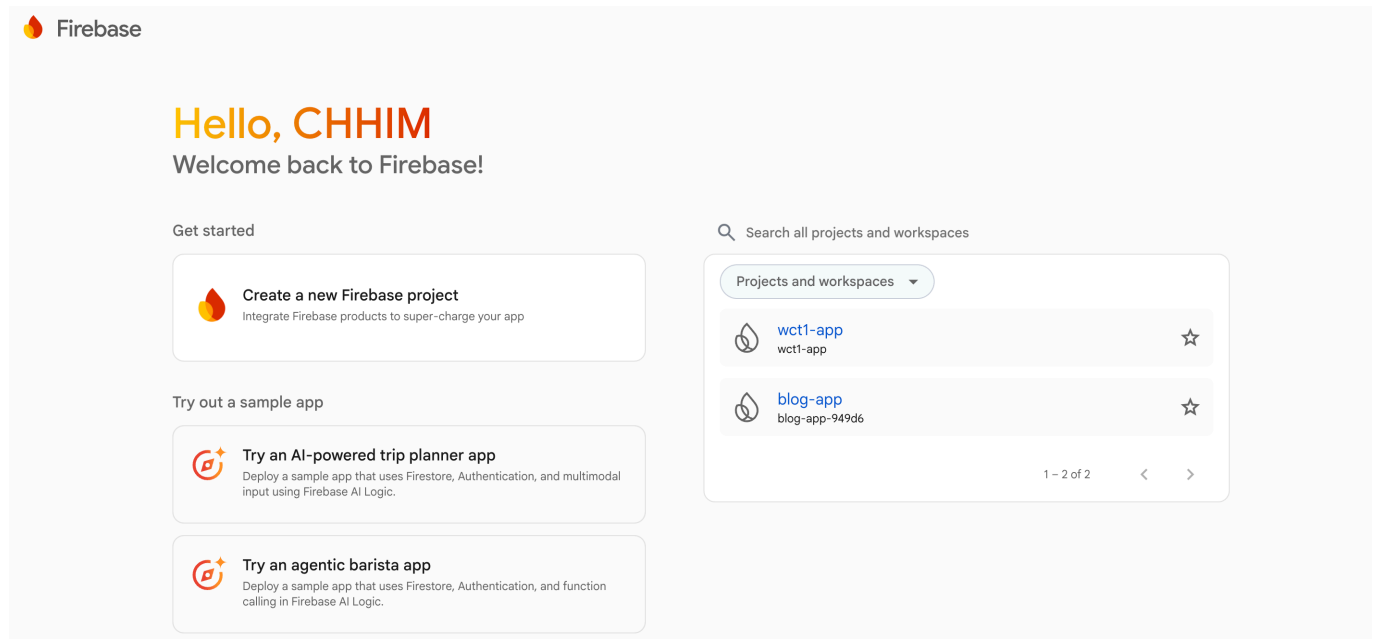
Lab – ReactJS part 2

Login & Register with Firebase (React + Vite)

Step 1: Create and Configure Your Firebase Project

Step 1.1: Step 1: Create the Project in Firebase Console

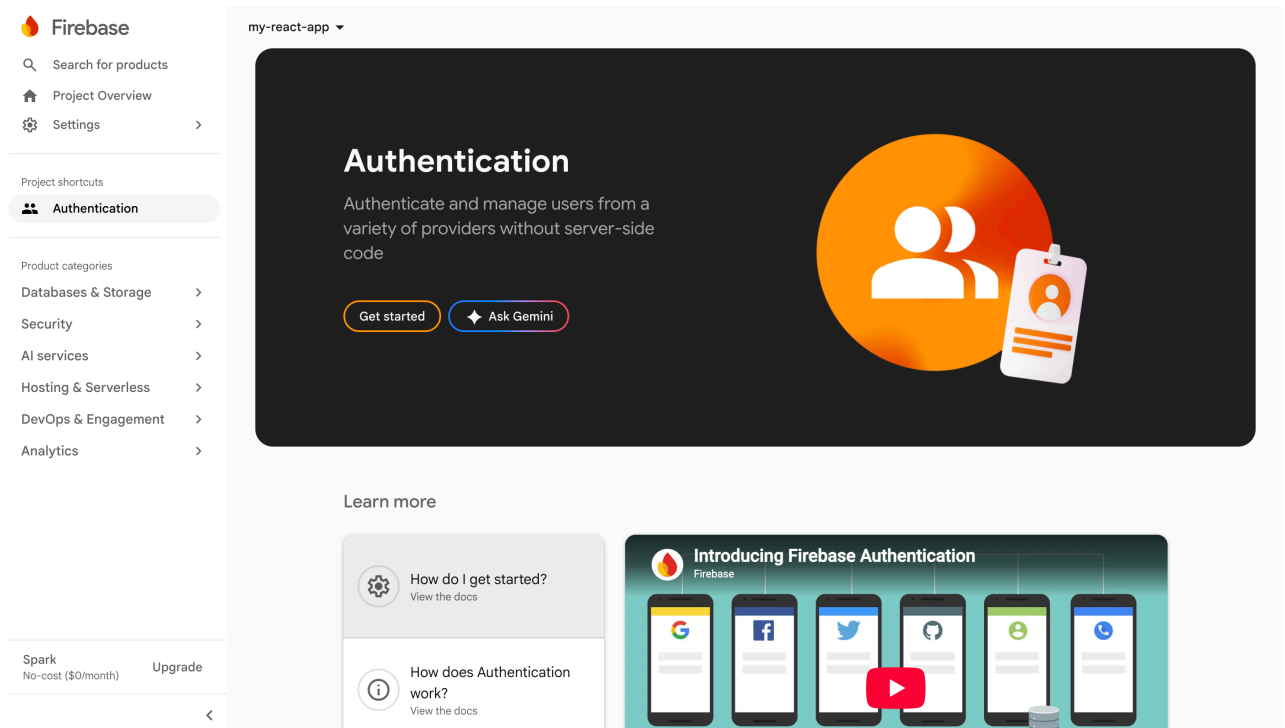
Go to: <https://console.firebase.google.com>, sign in with gmail account



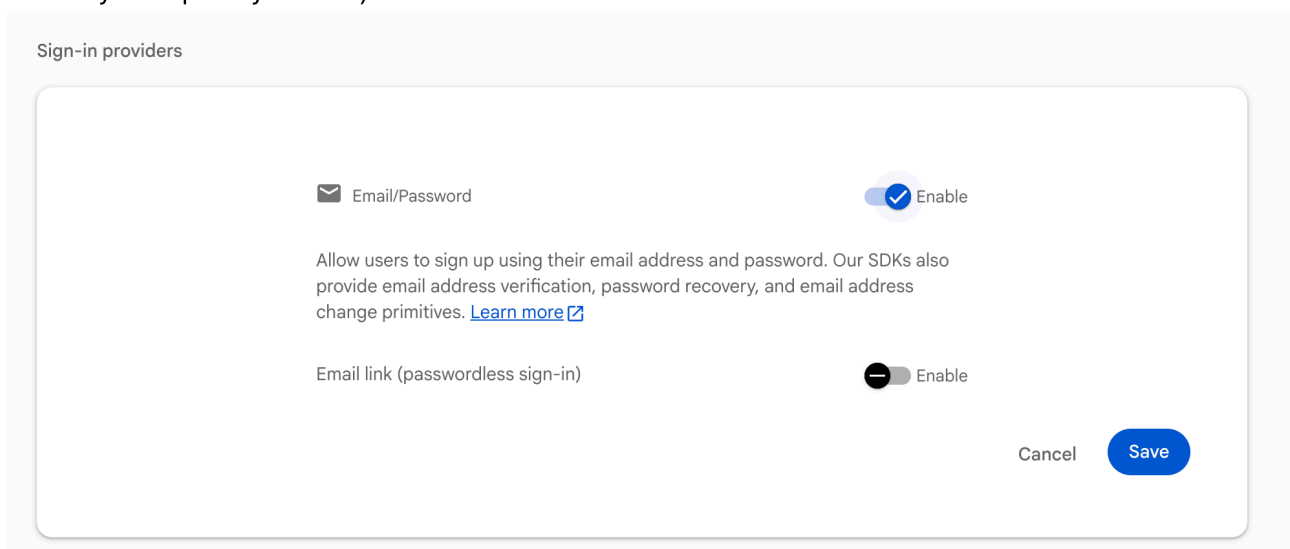
- Click "**Create a new Firebase project**"
- Enter a project name (e.g., my-react-app) and click Continue.
- Choose whether you want Gemini enabled (you can turn it off for a basic project), click **Continue**
- Choose whether you want Google Analytics enabled (you can turn it off for a basic project), then click **Create project**.
- Wait for the setup to complete and click Continue to enter your project dashboard.

Step 1.2: Enable Email & Password Authentication

- In the left-hand sidebar menu, click on **Security->Authentication**.



- Click the **Get started** button.
- Under the Sign-in method tab, look at the Native providers list and click on **Email/Password**.
- Toggle** the **Email/Password** switch to **Enabled**. (Leave Email link (passwordless sign-in) disabled unless you explicitly want it).



- Click **"Save"**

Step 1.3: Register Your Web App to Get Keys

- Go back to your **Project Overview dashboard** page

The screenshot shows the Firebase Project Overview dashboard for a project named 'my-react-app'. On the left is a sidebar with the Firebase logo, a search bar, and navigation links for 'Project Overview' (selected), 'Settings', 'Project shortcuts', 'Authentication', 'Product categories' (with sub-links for Databases & Storage, Security, AI services, Hosting & Serverless, DevOps & Engagement, and Analytics), and 'Spark' (No-cost (\$0/month) and Upgrade). The main content area has a top banner with a Gemini AI assistant tip. Below this, it says 'my-react-app' with a 'Spark plan' badge and a '+ Add app' button. A large 'Hello, CHHIM' greeting is followed by 'Welcome to your Firebase project!' and a note about Gemini suggestions. Under 'Next steps with Gemini', there are four cards: 'Tell us about your app' (highlighted with a yellow border), 'Add Analytics and monitoring', 'Build a backend', and 'Add AI to your app'. Each card has a brief description and a 'Learn more' link. A 'View docs' link is also present.

- Click "+ Add app" button -> "</>" button
- Enter an app nickname (e.g., My-React-Webapp) and click Register app.

The screenshot shows the 'Add Firebase to your web app' dialog. At the top, it says 'Add Firebase to your web app'. Below this is a purple box with a lightning bolt icon and the text 'You can also register a web app with an AI coding agent' and a 'Try it' button. The main section is titled '1 Register app'. It has a label 'App nickname' with a help icon, followed by a text input field containing 'My-React-Webapp'. Below the input field is a checkbox labeled 'Also set up Firebase Hosting for this app.' with a 'Learn more' link. A note states 'Hosting can also be set up later. There is no cost to get started anytime.' At the bottom of the first section is a blue 'Register app' button. Below this is the second section, '2 Add Firebase SDK'.

- Firebase will display an initialization code block containing a firebaseConfig object. It looks like this:

2 Add Firebase SDK

☒ Use npm ☐ Use a <script> tag

If you're already using [npm](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([Learn more](#)):

```
$ npm install firebase
```

Then, initialize Firebase and begin using the SDKs for the products you'd like to use.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyD4g0MR2WTiLKPPr484JzJQUdgosBp23Sc",
  authDomain: "my-react-app-7bc7e.firebaseio.com",
  projectId: "my-react-app-7bc7e",
  storageBucket: "my-react-app-7bc7e.firebaseio.com",
  messagingSenderId: "224019634053",
  appId: "1:224019634053:web:12ab66601a39e6748456ec"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
```

Note: This option uses the [modular JavaScript SDK](#), which provides reduced SDK size.

Learn more about Firebase for web: [Get Started](#), [Web SDK API Reference](#), [Samples](#)

Continue to console

- Keep this tab open or copy those values! You will need them for your local environment variables.

Step 2: Update Your Local React Project

Step 2.1: Save Keys in Your Environment Variables

Create/edit a file named `.env.local` in the root of your project.

```
VITE_FIREBASE_API_KEY="AIzaSyBUiuBN56ftA34WmQW7AJnHG98wJx4GgIE"
VITE_FIREBASE_AUTH_DOMAIN="wct1-app.firebaseio.com"
VITE_FIREBASE_PROJECT_ID="wct1-app"
VITE_FIREBASE_STORAGE_BUCKET="wct1-app.firebaseio.com"
VITE_FIREBASE_MESSAGING_SENDER_ID="1071075088616"
VITE_FIREBASE_APP_ID="1:1071075088616:web:fee80e3e6c0db8a4dd915d"
```

Step 2.2: Install the Firebase Package

```
npm install firebase
```

Step 2.3: Create Your Firebase Client File

Create a new file named **firebaseClient.js** inside your **src/lib/** folder:

File: **src/lib/firebaseClient.js**

```
// src/lib/firebaseClient.js
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
```

Step 3: App Component & Session Management

In File: **src/main.jsx**

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

In File: **src/App.jsx**

```
// src/App.jsx
import { useEffect, useState } from "react";
import { Routes, Route, Navigate } from "react-router-dom";
import { auth } from "../lib/firebaseClient";
import { onAuthStateChanged, signOut } from "firebase/auth";
import Navbar from "../components/Navbar";
import Sidebar from "../components/Sidebar";
import Footer from "../components/Footer";
import HomePage from "../pages/HomePage";
import LoginPage from "../pages/LoginPage";
import RegisterPage from "../pages/RegisterPage";

export default function App() {
  const [user, setUser] = useState(null);
  const [checkingSession, setCheckingSession] = useState(true);

  // Set default to true so it's open on desktop initially, but fully
  // toggleable!
  const [isSidebarOpen, setIsSidebarOpen] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
      setUser(currentUser || null);
      setCheckingSession(false);
      // Auto-close or open based on login status
      setIsSidebarOpen(!!currentUser);
    });
    return () => unsubscribe();
  }, []);

  async function handleLogout() {
    try {
      await signOut(auth);
      setIsSidebarOpen(false);
    } catch (err) {
      console.error("Error signing out:", err);
    }
  }

  if (checkingSession) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-gray-50">
        <p className="text-gray-600 animate-pulse font-medium">Checking
        session...</p>
      </div>
    );
  }

  return (
    <div className="flex flex-col min-h-screen bg-gray-50 text-gray-800">
      { /* Top Header Control Banner */ }
    </div>
  );
}
```

```

    <Navbar
      user={user}
      onLogout={handleLogout}
      isSidebarOpen={isSidebarOpen}
      setIsSidebarOpen={setIsSidebarOpen}
    />

    <div className="flex flex-1 pt-16 relative">
      {/* Left Off-Canvas Sidebar Panels */}
      {user && (
        <Sidebar
          user={user}
          isOpen={isSidebarOpen}
          setIsOpen={setIsSidebarOpen}
        />
      )}

      {/* Dynamic Content Viewport - Padding shifts based on sidebar
toggle status */}
      <main className={`flex-1 p-4 md:p-6 transition-all duration-300
${user && isSidebarOpen ? 'md:pl-64' : 'md:pl-0'}}`>
        <Routes>
          <Route path="/" element={<HomePage user={user} />} />
          <Route
            path="/login"
            element={!user ? <LoginPage /> : <Navigate to="/" replace />}
          />
          <Route
            path="/register"
            element={!user ? <RegisterPage /> : <Navigate to="/" replace
/>}
          />
        </Routes>
      </main>
    </div>

    {/* Footer adjustments to match current layout space */}
    <div className={`transition-all duration-300 ${user && isSidebarOpen
? 'md:pl-64' : 'md:pl-0'}}`>
      <Footer />
    </div>
  </div>
);
}

```

Step 4. Implement Pages

4.1 Home Page

File `src/pages/HomePage.jsx`

```

//src/pages/HomePage.jsx
import profilePic from "../assets/profile.jpg";
import StemBuilding from "../assets/stem.jpeg";
import { StatCard, TableRow } from "../components/Dashboard";

export default function HomePage({ user }) {
  // ===== DASHBOARD =====
  if (user) {
    return (
      <main className="bg-gray-50">
        <div className="max-w-6xl mx-auto space-y-8">

          {/* Header */}
          <div className="flex items-center justify-between bg-white p-6
rounded-2xl shadow-sm border">
            <div>
              <h1 className="text-2xl md:text-3xl font-bold text-gray-800">
                Dashboard
              </h1>
              <p className="text-sm text-gray-500 mt-1">
                Welcome back, {user.name}
                <span className="font-medium">{user.email}</span>
              </p>
            </div>

            <div className="w-12 h-12 rounded-full bg-gray-600 text-white
flex items-center justify-center font-semibold text-lg shadow-sm">
              {user.email.charAt(0).toUpperCase()}
            </div>
          </div>

          {/* Stats */}
          <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4
gap-4">
            <StatCard title="Total Projects" value="12" />
            <StatCard title="Active" value="8" color="text-green-600" />
            <StatCard title="Pending" value="3" color="text-yellow-600" />
            <StatCard title="Inactive" value="1" color="text-red-600" />
          </div>

          {/* Table */}
          <div className="bg-white rounded-2xl shadow-sm border overflow-
hidden">
            <div className="px-6 py-4 border-b flex items-center justify-
between">
              <h2 className="text-lg font-semibold text-gray-800">
                Project Overview
              </h2>
              <span className="text-xs text-gray-500 uppercase">
                Sample Data
              </span>
            </div>

```



```

        <div className="overflow-x-auto">
          <table className="min-w-full text-sm">
            <thead className="bg-gray-50 text-gray-600">
              <tr>
                <th className="px-6 py-3 text-left font-medium">ID</th>
                <th className="px-6 py-3 text-left font-
medium">Name</th>
                <th className="px-6 py-3 text-left font-
medium">Status</th>
              </tr>
            </thead>
            <tbody className="divide-y">
              <TableRow id="1" name="Sample Item A" status="Active" />
              <TableRow id="2" name="Sample Item B" status="Pending" />
              <TableRow id="3" name="Sample Item C" status="Inactive"
/>
            </tbody>
          </table>
        </div>
      </div>
    </main>
  );
}

// ===== PUBLIC PAGE =====
return (
  <main className="flex-1 flex flex-col items-center justify-center
space-y-6">
    <section className="flex text-start px-4 space-x-4">
      <img
        src={profilePic}
        alt="Profile"
        className="w-40 h-40 rounded-full shadow-lg border-4 border-
white"
      />
      <div className="space-y-4">
        <h1 className="text-4xl md:text-5xl font-bold">
          Welcome to My React Site
        </h1>
        <p className="text-lg md:text-xl">
          Hello! I'm learning{" "}
          <span className="font-semibold text-blue-600">React</span>.
        </p>
      </div>
    </section>

    <img src={StemBuilding} alt="StemBuilding" className="max-w-3xl" />
  </main>

```

```
);  
}
```

4.2 Register Page (sign up)

To allows users to create a new account.

File: `src/pages/RegisterPage.jsx`

```
// src/pages/RegisterPage.jsx  
import { useState } from "react";  
import { Link, useNavigate } from "react-router-dom";  
import { auth } from "../lib/firebaseClient";  
import { createUserWithEmailAndPassword } from "firebase/auth";  
  
export default function RegisterPage() {  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  const [confirmPassword, setConfirmPassword] = useState("");  
  const [loading, setLoading] = useState(false);  
  const [error, setError] = useState(null);  
  
  const navigate = useNavigate();  
  
  async function handleRegister(e) {  
    e.preventDefault();  
    setError(null);  
  
    if (password !== confirmPassword) {  
      setError("Passwords do not match");  
      return;  
    }  
  
    if (password.length < 6) {  
      setError("Password must be at least 6 characters");  
      return;  
    }  
  
    setLoading(true);  
  
    try {  
      await createUserWithEmailAndPassword(auth, email, password);  
      navigate("/"); // Firebase logs users in directly after signup. Go  
home!  
    } catch (authError) {  
      console.error("Firebase registration error:", authError);  
      setError(authError.message.replace("Firebase: ", ""));  
    } finally {  
      setLoading(false);  
    }  
  }  
}
```

```

    return (
      <section className="flex-1 flex items-center justify-center bg-gray-50
px-4 py-12">
        <div className="w-full max-w-md bg-white rounded-2xl border border-
gray-200 p-8 shadow-sm">
          <h1 className="text-3xl font-bold mb-1 text-center text-gray-
800">Register</h1>
          <p className="text-center text-sm text-gray-500 mb-6">Create a new
account</p>

          <form onSubmit={handleRegister} className="space-y-5">
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-
1">Email Address</label>
              <input
                type="email"
                value={email}
                disabled={loading}
                onChange={(e) => setEmail(e.target.value)}
                placeholder="you@example.com"
                required
                className="w-full rounded-lg border border-gray-300 px-4 py-
2.5 text-sm focus:outline-none focus:ring-2 focus:ring-blue-600
focus:border-blue-600"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700 mb-
1">Password</label>
              <input
                type="password"
                value={password}
                disabled={loading}
                onChange={(e) => setPassword(e.target.value)}
                placeholder="••••••••"
                required
                className="w-full rounded-lg border border-gray-300 px-4 py-
2.5 text-sm focus:outline-none focus:ring-2 focus:ring-blue-600
focus:border-blue-600"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700 mb-
1">Confirm Password</label>
              <input
                type="password"
                value={confirmPassword}
                disabled={loading}
                onChange={(e) => setConfirmPassword(e.target.value)}
                placeholder="••••••••"
                required

```

```

        className="w-full rounded-lg border border-gray-300 px-4 py-
2.5 text-sm focus:outline-none focus:ring-2 focus:ring-blue-600
focus:border-blue-600"
      />
    </div>

    {error && <p className="text-red-600 text-center text-sm">{error}</p>}}

    <button
      type="submit"
      disabled={loading}
      className="w-full bg-blue-600 text-white py-2.5 rounded-lg
font-semibold hover:bg-blue-700 active:scale-[0.99] transition-all
disabled:bg-gray-400"
    >
      {loading ? "Creating account..." : "Register"}
    </button>
  </form>

  <p className="mt-6 text-center text-sm text-gray-600">
    Already have an account?{" "}
    <Link to="/login" className="font-medium text-blue-600
hover:underline">
      Login
    </Link>
  </p>
</div>
</section>
);
}

```

4.3. Implement Login Page (sign in)

To allows registered users to log in.

File: `src/pages/LoginPage.jsx`

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { auth } from "../lib/firebaseClient";
import { signInWithEmailAndPassword } from "firebase/auth";

export default function LoginPage() {
  const [email, setEmail] = useState(""); // form state
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false); // loading state
  const [error, setError] = useState(null); // error message state

  const navigate = useNavigate();

```

```

    async function handleLogin(e) {
      e.preventDefault();
      setError(null);
      setLoading(true);

      try {
        await signInWithEmailAndPassword(auth, email, password);
        navigate("/"); // The App.jsx global listener automatically handles
state update!
      } catch (authError) {
        console.error("Firebase login error:", authError);
        setError(authError.message.replace("Firebase: ", ""));
      } finally {
        setLoading(false);
      }
    }

    return (
      <section className="flex-1 flex items-center justify-center bg-gray-50
px-4 py-12">
        <div className="w-full max-w-md bg-white rounded-2xl border border-
gray-200 p-8 shadow-sm">
          <h2 className="text-3xl font-bold text-center text-gray-800 mb-
1">Welcome Back</h2>
          <p className="text-center text-sm text-gray-500 mb-6">Login to your
account</p>

          <form className="space-y-5" onSubmit={handleLogin}>
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-
1">Email Address</label>
              <input
                type="email"
                placeholder="you@example.com"
                value={email}
                disabled={loading}
                onChange={(e) => setEmail(e.target.value)}
                required
                className="w-full rounded-lg border border-gray-300 px-4 py-
2.5 text-sm focus:outline-none focus:ring-2 focus:ring-blue-600
focus:border-blue-600"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700 mb-
1">Password</label>
              <input
                type="password"
                placeholder="....."
                value={password}
                disabled={loading}
                onChange={(e) => setPassword(e.target.value)}
                required

```

```

        className="w-full rounded-lg border border-gray-300 px-4 py-
2.5 text-sm focus:outline-none focus:ring-2 focus:ring-blue-600
focus:border-blue-600"
      />
    </div>

    {error && <p className="text-sm text-red-600 text-center">{error}</p>}}

    <button
      type="submit"
      disabled={loading}
      className="w-full rounded-lg bg-blue-600 py-2.5 text-white
font-semibold hover:bg-blue-700 active:scale-[0.99] transition-all
disabled:bg-gray-400"
    >
      {loading ? "Signing in..." : "Sign In"}
    </button>
  </form>

  <p className="mt-6 text-center text-sm text-gray-600">
    Don't have an account?{" "}
    <Link to="/register" className="font-medium text-blue-600
hover:underline">
      Register
    </Link>
  </p>
</div>
</section>
);
}

```

Step 5: Components

5.1 Navbar Component

The Navbar updates based on login status. **File:** `src/components/Navbar.jsx`

```

// src/components/Navbar.jsx
import { Link } from "react-router-dom";

export default function Navbar({ user, onLogout, isSidebarOpen,
setIsSidebarOpen }) {
  return (
    <header className="bg-white border-b border-gray-200 fixed top-0 left-0
right-0 h-16 z-50">
      <nav className="h-full px-4 flex items-center justify-between">

        {/* Left Side Group: Logo first, then the Toggle Button */}
        <div className="flex items-center gap-10">
          <Link to="/" className="text-xl font-bold text-blue-600 tracking-

```

```

tight">
    My Web App
  </Link>

  {user && (
    <button
      onClick={() => setIsSidebarOpen(!isSidebarOpen)}
      className="p-2 rounded-lg hover:bg-gray-100 text-gray-600
transition-colors focus:outline-none"
      aria-label="Toggle Navigation Drawer"
    >
      <svg className="w-6 h-6" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
        {isSidebarOpen ? (
          // Close Icon Shape
          <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M6 18L18 6M6 6L12 12" />
        ) : (
          // Menu Hamburger Icon Shape
          <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M4 6h16M4 12h16M4 18h16" />
        )}
      </svg>
    </button>
  )}
</div>

{/* Right Side Group: User Action Links */}
<div className="flex items-center gap-4">
  {!user ? (
    <div className="flex items-center gap-2">
      <Link to="/register" className="px-3 py-1.5 rounded-md text-
sm font-medium text-gray-600 hover:bg-gray-100 transition-colors">
        Register
      </Link>
      <Link to="/login" className="px-3 py-1.5 rounded-md text-sm
font-semibold bg-blue-600 text-white hover:bg-blue-700 transition-colors
shadow-sm">
        Login
      </Link>
    </div>
  ) : (
    <div className="flex items-center gap-4">
      <span className="text-sm font-medium text-gray-500 hidden
sm:inline">
        {user.email}
      </span>
      <button
        onClick={onLogout}
        className="px-3 py-1.5 rounded-md text-sm font-semibold bg-
red-600 text-white hover:bg-red-500 transition-colors shadow-sm"
      >
        Logout
      </button>
    </div>
  )}
  </div>

```

5.2 Footer Component

```
export default function Footer() {
  return (
    <footer>
      <div className="max-w-6xl mx-auto px-4 py-6 flex items-center justify-center">
        <p className="text-sm ">
          © {new Date().getFullYear()} MyReactSite. All rights reserved.
        </p>
      </div>
    </footer>
  );
}
```

File src/components/Sidebar.jsx

16 / 18


```

    return (
      <>
        {/* Mobile-only Background Overlay Backdrop Mask (Dismisses layout
panel when tapping outside) */}
        {isOpen && (
          <div
            className="fixed inset-0 bg-gray-900/30 backdrop-blur-sm z-40
md:hidden"
            onClick={() => setIsOpen(false)}
          />
        )}

        {/* Toggleable Drawer Viewport Container Panel */}
        <aside
          className={`fixed top-16 bottom-0 left-0 w-64 bg-white border-r
border-gray-200 z-40 p-4 flex flex-col justify-between transition-transform
duration-300 ${
            isOpen ? 'translate-x-0' : '-translate-x-full'
          }`}
        >
          <div className="space-y-6">
            {/* Menu Category Info Text */}
            <div className="px-4 text-xs font-semibold text-gray-400
uppercase tracking-wider">
              Admin Management
            </div>

            {/* Core App Navigation Link Group */}
            <nav className="space-y-1">
              {menuItems.map((item, index) => (
                <Link
                  key={index}
                  to={item.path}
                  // Only trigger automatic close on small touch devices
                  onClick={() => {
                    if (window.innerWidth < 768) setIsOpen(false);
                  }}
                  className={location.pathname === item.path ? activeClass :
linkClass}
                >
                  {item.name}
                </Link>
              ))}
            </nav>
          </div>

          {/* Bottom Profile Details Row */}
          <div className="border-t border-gray-200 pt-4 flex items-center
gap-3">
            <div className="w-9 h-9 rounded-full bg-blue-100 text-blue-600
flex items-center justify-center font-bold text-sm select-none">
              {user?.email?.charAt(0).toUpperCase()}
            </div>
            <div className="truncate max-w-[160px]">

```

```

        <p className="text-xs font-semibold text-gray-800 truncate">
{user?.email}</p>
        <p className="text-[10px] text-gray-400 font-medium">System
Administrator</p>
    </div>
</div>
</aside>
</>
);
}

```

5.4 Dashboard Component

Files: `src/components/Dashboard.jsx`

```

export function StatCard({ title, value, color = "text-blue-600" }) {
  return (
    <div className="bg-white rounded-xl border shadow-sm p-5">
      <p className="text-sm text-gray-500">{title}</p>
      <p className={`text-3xl font-bold mt-2 ${color}`}>{value}</p>
    </div>
  );
}

export function TableRow({ id, name, status }) {
  const statusColor =
    status === "Active"
      ? "text-green-600"
      : status === "Pending"
      ? "text-yellow-600"
      : "text-red-600";

  return (
    <tr>
      <td className="px-6 py-3">{id}</td>
      <td className="px-6 py-3">{name}</td>
      <td className={`px-6 py-3 font-medium ${statusColor}`}>
        {status}
      </td>
    </tr>
  );
}

```

Note: please follow previous lab to install `tailwindcss` and `react-router`

- Install Tailwind CSS `npm install tailwindcss @tailwindcss/vite`
- Install react-router `npm install react-router-dom`
- Git repository: https://github.com/rupp-ite/wct1_reactjs/tree/part2